

01_base_sri

May 9, 2026

1 Feature Engineering — Capa 1: Datos SRI

Objetivo: Construir las 8 features base del modelo de clustering a partir del cruce entre `match_final_empresas.csv` y los archivos SRI provinciales.

1.1 Flujo de este notebook

1. Cargar `match_final_empresas.csv` (177 empresas con RUC verificado)
2. Consolidar los 26 archivos SRI provinciales en un solo DataFrame
3. JOIN izquierdo: `match_final` $\leftarrow$ SRI por RUC
4. Construir las 8 features (binarias, numéricas, categóricas)
5. Exportar `features_capa1.csv`

Nota: `es_cliente_fpa` viaja como columna de referencia pero **NO se usa** en el modelo de clustering. Es solo para validación externa posterior.

```
[1]: # Imports
import pandas as pd
import numpy as np
import glob
import warnings
from pathlib import Path

warnings.filterwarnings('ignore')

# Rutas base
ROOT = Path('e:/TESIS MAESTRIA/Desarrollo_clustering_maestria')
PATH_MATCH = ROOT / '02_data_cleaning/outputs/match_final_empresas.csv'
PATH_SRI = ROOT / '02_data_cleaning/data_SRI'
PATH_OUTPUT = ROOT / '03_feature_engineering/outputs'
PATH_OUTPUT.mkdir(parents=True, exist_ok=True)

print('Rutas configuradas correctamente.')
```

Rutas configuradas correctamente.

1.2 1. Cargar match_final_empresas.csv

```
[2]: # Cargamos el archivo base con los 177 leads que tienen RUC verificado.
match = pd.read_csv(PATH_MATCH, dtype={'RUC': str})

# Limpiar RUC: eliminar sufijo '.0' (proviene de float al leer CSV), luego
↳ zfill a 13 dígitos
match['RUC'] = (
    match['RUC']
    .astype(str)
    .str.strip()
    .str.replace(r'\.0$', '', regex=True) # elimina el .0 del float
    .str.zfill(13)
)

# Paso 1 - Agregar es_cliente_fpa
# Las empresas de fuente HORAS provienen de proyectos_empresa.xlsx (clientes
↳ reales de FPA).
# Las empresas de fuente LEADS son prospectos del CRM.
# No existen RUCs compartidos entre ambas fuentes, por lo que la derivación es
↳ directa.
match['es_cliente_fpa'] = (match['source_label'] == 'HORAS').astype(int)

print(f'Empresas en match_final: {len(match)}')
print(f'Columnas: {list(match.columns)}')
print(f'Clientes FPA en dataset: {match["es_cliente_fpa"].sum()} /
↳ {len(match)}')
print(f'Longitud RUC (debe ser todo 13): {match["RUC"].str.len().value_counts().
↳ to_dict()}')
print()
print(match[['name_norm', 'RUC', 'source_winner', 'es_cliente_fpa']].head(5).
↳ to_string())
```

Empresas en match_final: 169

Columnas: ['source_label', 'name_raw', 'name_norm', 'RUC', 'source_winner',
'score', 'verdict', 'es_cliente_fpa']

Clientes FPA en dataset: 54 / 169

Longitud RUC (debe ser todo 13): {13: 169}

	name_norm	RUC	source_winner	es_cliente_fpa
0	7 ELEVEN MEXICO	1400612998001	SRI_FANTASIA	0
1	ABBOTT	0990000670001	SCVS	0
2	ABBVIE	1792489156001	SRI_RAZON	0
3	AFP GENESIS	0991307605001	SCVS	0
4	AKROS	1791148800001	SCVS_EXACT	0

1.3 2. Consolidar los 26 archivos SRI provinciales

Los archivos SRI tienen: - Separador: | - Encoding: utf-8-sig - Columna RUC: NUMERO_RUC

Se consolidan todos y se deduplica por RUC (keep='first') porque una empresa puede tener establecimientos registrados en más de una provincia.

```
[3]: # Cargar y consolidar todos los archivos SRI provinciales
sri_files = glob.glob(str(PATH_SRI / 'SRI_RUC_*.csv'))
print(f'Archivos SRI encontrados: {len(sri_files)}')

chunks = []
for f in sri_files:
    try:
        df_chunk = pd.read_csv(f, sep='|', encoding='utf-8-sig', dtype=str,
                                low_memory=False)
        chunks.append(df_chunk)
    except Exception as e:
        print(f'ERROR {f}: {e}')

sri = pd.concat(chunks, ignore_index=True)
sri.columns = [c.strip() for c in sri.columns]
sri['NUMERO_RUC'] = sri['NUMERO_RUC'].astype(str).str.strip()

# Deduplicar: si una empresa aparece en 2+ provincias, conservar la primera
# aparición
sri_dedup = sri.drop_duplicates(subset='NUMERO_RUC', keep='first').copy()

print(f'Total registros SRI consolidados: {len(sri):,}')
print(f'Registros únicos por RUC (deduplicados): {len(sri_dedup):,}')
print(f'Columnas: {list(sri_dedup.columns)}')
```

Archivos SRI encontrados: 24

Total registros SRI consolidados: 8,050,507

Registros únicos por RUC (deduplicados): 6,799,570

Columnas: ['NUMERO_RUC', 'RAZON_SOCIAL', 'CODIGO_JURISDICCION', 'ESTADO_CONTRIBUYENTE', 'CLASE_CONTRIBUYENTE', 'FECHA_INICIO_ACTIVIDADES', 'FECHA_ACTUALIZACION', 'FECHA_SUSPENSION_DEFINITIVA', 'FECHA_REINICIO_ACTIVIDADES', 'OBLIGADO', 'TIPO_CONTRIBUYENTE', 'NUMERO_ESTABLECIMIENTO', 'NOMBRE_FANTASIA_COMERCIAL', 'ESTADO_ESTABLECIMIENTO', 'DESCRIPCION_PROVINCIA_EST', 'DESCRIPCION_CANTON_EST', 'DESCRIPCION_PARROQUIA_EST', 'CODIGO_CIIU', 'ACTIVIDAD_ECONOMICA', 'AGENTE_RETENCION', 'ESPECIAL']

1.4 3. JOIN match_final ← SRI por RUC

Se usa **LEFT JOIN** para conservar las 177 empresas aunque alguna no se encuentre en el catálogo SRI. Las empresas sin match quedarán con NaN en las columnas SRI y serán excluidas del dataset final de features (documentadas como limitación).

```
[4]: # JOIN izquierdo: match_final (177) + SRI por RUC
df = match.merge(
    sri_dedup,
    left_on='RUC',
    right_on='NUMERO_RUC',
    how='left'
)

# Diagnóstico del join
con_datos_sri = df['NUMERO_RUC'].notna().sum()
sin_datos_sri = df['NUMERO_RUC'].isna().sum()

print(f'Total empresas tras JOIN: {len(df)}')
print(f'Con datos SRI: {con_datos_sri}')
print(f'Sin datos SRI (excluir): {sin_datos_sri}')
print()

if sin_datos_sri > 0:
    print('Empresas SIN match en SRI (se excluirán del modelo):')
    print(df[df['NUMERO_RUC'].isna()][['name_norm', 'RUC', 'source_winner',
    ↪ 'es_cliente_fpa']].to_string())
```

```
Total empresas tras JOIN: 169
Con datos SRI: 167
Sin datos SRI (excluir): 2
```

Empresas SIN match en SRI (se excluirán del modelo):

	name_norm	RUC	source_winner	es_cliente_fpa
44	EQUIVIDA	1091798976001	SCVS	1
148	SMI	1391938615001	SCVS	0

1.5 4. Construcción de las 8 features

#	Feature	Columna SRI origen	Transformación
1	tipo_sociedad	TIPO_CONTRIBUYENTE	SOCIEDAD=1, otro=0
2	obligado_contabilidad	OBLIGADO	S=1, N=0
3	es_agente_retencion	AGENTE_RETENCION	S=1, N=0
4	es_contribuyente_especial	ESPECIAL	S=1, N=0
5	estado_activo	ESTADO_CONTRIBUYENTE	ACTIVO=1, otro=0
6	antiguedad_anos	FECHA_INICIO_ACTIVIDAD	2026 - año(fecha)
7	sector_ciiu_macro	CODIGO_CIIU	Primera letra → G/C/M/K/S/OTRO
8	region	DESCRIPCION_PROVINCIA	EST. Incha/Guayas/Resto

```

[5]: # Feature 1: tipo_sociedad
# SOCIEDAD = empresa jurídica formalmente constituida (SA, Ltda, etc.)
# PERSONA NATURAL = persona natural con RUC y actividad económica
df['tipo_sociedad'] = (df['TIPO_CONTRIBUYENTE'].str.strip().str.upper() == 'SOCIEDAD').astype(int)

# Feature 2: obligado_contabilidad
# Empresa obligada a llevar contabilidad formal.
# El SRI obliga a empresas con ingresos > 300K USD anuales o activos > 180K USD.
# Es un proxy indirecto de tamaño económico mínimo.
df['obligado_contabilidad'] = (df['OBLIGADO'].str.strip().str.upper() == 'S').astype(int)

# Feature 3: es_agente_retencion
# Autorizado por el SRI para retener impuestos en sus pagos a proveedores.
# Indica mayor volumen de transacciones y relaciones comerciales activas.
df['es_agente_retencion'] = (df['AGENTE_RETENCION'].str.strip().str.upper() == 'S').astype(int)

# Feature 4: es_contribuyente_especial
# Designación SRI para empresas de gran tamaño fiscal (mayor control tributario).
# En Ecuador es un indicador de empresa grande o con alta relevancia económica.
df['es_contribuyente_especial'] = (df['ESPECIAL'].str.strip().str.upper() == 'S').astype(int)

# Feature 5: estado_activo
# Empresa activa (en operación) vs suspendida o pasiva.
# Relevante para cualificar si el lead es un prospecto válido.
df['estado_activo'] = (df['ESTADO_CONTRIBUYENTE'].str.strip().str.upper() == 'ACTIVO').astype(int)

# Feature 6: antigüedad_anos
# Madurez del negocio: empresas más antiguas tienen estructuras más consolidadas
# y mayor propensión a invertir en proyectos tecnológicos.
# Formato real en SRI: 'YYYY-MM-DD HH:MM:SS' → se parsea sin formato explícito
df['FECHA_INICIO_ACTIVIDADES'] = pd.to_datetime(
    df['FECHA_INICIO_ACTIVIDADES'], errors='coerce'
)
df['antigüedad_anos'] = 2026 - df['FECHA_INICIO_ACTIVIDADES'].dt.year
# Clip: eliminar valores negativos (fechas futuras en el registro SRI = error de datos)
df['antigüedad_anos'] = df['antigüedad_anos'].clip(lower=0)

# Feature 7: sector_ciiu_macro
# Primera letra del código CIIU = sector económico a nivel macro.

```

```

# G=Comercio, C=Manufactura, M=Profesional/Técnico, K=Financiero, S=Servicios,
↳OTRO
SECTORES_PRINCIPALES = {'G', 'C', 'M', 'K', 'S'}
df['sector_ciiu_macro'] = (
    df['CODIGO_CIIU'].astype(str).str.strip().str[0].str.upper()
)
df['sector_ciiu_macro'] = df['sector_ciiu_macro'].where(
    df['sector_ciiu_macro'].isin(SECTORES_PRINCIPALES), other='OTRO'
)

# Feature 8: region
# Las dos provincias con mayor concentración de empresas FPA; el resto agrupado.
def asignar_region(prov):
    if pd.isna(prov):
        return None
    prov_upper = str(prov).strip().upper()
    if 'PICHINCHA' in prov_upper:
        return 'Pichincha'
    elif 'GUAYAS' in prov_upper:
        return 'Guayas'
    else:
        return 'Resto'

df['region'] = df['DESCRIPCION_PROVINCIA_EST'].apply(asignar_region)

# Resumen de distribuciones
print('=== Distribuciones de las 8 features ===')
print()
BINARIAS = ['tipo_sociedad', 'obligado_contabilidad', 'es_agente_retencion',
            'es_contribuyente_especial', 'estado_activo']
for col in BINARIAS:
    dist = df[col].value_counts(dropna=False).to_dict()
    print(f' {col}: {dist}')
print()
antig = df['antiguedad_anos']
print(f' antiguedad_anos: min={antig.min():.0f}, mediana={antig.median():.0f},
↳'
      f'max={antig.max():.0f}, nulos={antig.isna().sum()}')
print()
print(f' sector_ciiu_macro: {df["sector_ciiu_macro"].
↳value_counts(dropna=False).to_dict()}')
print(f' region: {df["region"].value_counts(dropna=False).
↳to_dict()}')

```

=== Distribuciones de las 8 features ===

tipo_sociedad: {1: 124, 0: 45}

```

obligado_contabilidad: {1: 122, 0: 47}
es_agente_retencion: {0: 110, 1: 59}
es_contribuyente_especial: {0: 118, 1: 51}
estado_activo: {1: 118, 0: 51}

antiguedad_anos: min=0, mediana=24, max=91, nulos=2

sector_ciiu_macro: {'G': 66, 'OTRO': 48, 'C': 27, 'M': 12, 'K': 9, 'S': 7}
region:           {'Guayas': 61, 'Pichincha': 54, 'Resto': 52, nan: 2}

```

1.6 5. Exportar features_capa1.csv

El archivo exportado contiene: - **Columnas de identidad:** name_norm, RUC, source_label, source_winner - **Label de validación:** es_cliente_fpa (NO entra al modelo de clustering) - 8 **features:** listas para encoding y escalado en el notebook 03_matriz_final.ipynb

```

[6]: COLS_ID      = ['name_norm', 'RUC', 'source_label', 'source_winner',
    ↪ 'es_cliente_fpa']
COLS_FEATURES = [
    'tipo_sociedad',
    'obligado_contabilidad',
    'es_agente_retencion',
    'es_contribuyente_especial',
    'estado_activo',
    'antiguedad_anos',
    'sector_ciiu_macro',
    'region'
]

# Solo empresas CON datos SRI (las sin match se excluyen del modelo)
features_capa1 = df[df['NUMERO_RUC'].notna()][COLS_ID + COLS_FEATURES].copy()
features_capa1 = features_capa1.reset_index(drop=True)

# Diagnóstico de nulos en las features
print('=== Diagnóstico de nulos por feature ===')
for col in COLS_FEATURES:
    n_null = features_capa1[col].isna().sum()
    estado = 'OK' if n_null == 0 else f'{n_null} NULOS'
    print(f' {col}: {estado}')

print()
print(f'Shape final features_capa1:      {features_capa1.shape}')
print(f'Clientes FPA en dataset:          {features_capa1["es_cliente_fpa"]}')
    ↪ sum() / {len(features_capa1)}')
print(f'Distribución es_cliente_fpa:     {features_capa1["es_cliente_fpa"]}')
    ↪ value_counts().to_dict()')

# Exportar

```

```
out_path = PATH_OUTPUT / 'features_capa1.csv'
features_capa1.to_csv(out_path, index=False)
print(f'\nExportado: {out_path}')
```

=== Diagnóstico de nulos por feature ===

```
tipo_sociedad: OK
obligado_contabilidad: OK
es_agente_retencion: OK
es_contribuyente_especial: OK
estado_activo: OK
antiguedad_anos: OK
sector_ciiu_macro: OK
region: OK
```

```
Shape final features_capa1:      (167, 13)
Clientes FPA en dataset:         53 / 167
Distribución es_cliente_fpa:     {0: 114, 1: 53}
```

```
Exportado: e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\03_feature_engineeri
ng\outputs\features_capa1.csv
```